Co-Op
Enterprise AI OS

Master Build Prompt + Complete Task Registry

Use this document to brief any AI coding agent and track every build task

Document	Master Build Prompt + 52-Task Production Registry
Reference Doc	CoOp_Full_Engineering_Documentation_v4.0.docx
AI Agent Target	Any LLM coding agent: Claude, Cursor, Copilot, Windsurf, etc.
Phase 0 Target	Company GPT running in 7 days — 6 services, Docker Compose
Full Platform	12-month roadmap — 30 services, Kubernetes, KubeRay
Total Tasks	52 production-ready tasks across 5 phases
Stack	Next.js · FastAPI · LangGraph · Temporal · Qdrant · LiteLLM · vLLM
Architecture	Progressive: 6 → 12 → 20 → 30 services

How to Use This Document

PART 1 (Page 2): Copy the Master Prompt verbatim into any AI coding agent — Claude, Cursor, GitHub Copilot, Windsurf, or similar. The prompt instructs the agent to read the attached engineering documentation and begin building Phase 0 immediately.

PART 2 (Pages 4+): The complete task registry — 52 tasks across 5 phases with step-by-step instructions, time estimates, dependencies, and acceptance criteria for every single task.

Start with Task T-00 through T-09 (Phase 0). Only move to Phase 1 after ALL Phase 0 acceptance criteria pass.

PART 1 — Master Build Prompt

Copy the entire prompt below and paste it into your AI coding agent (Claude, Cursor, Windsurf, GitHub Copilot, or any code-generation LLM). Attach the engineering documentation file as a reference when starting.

—— COPY FROM HERE ——————————————————————————————

SYSTEM ROLE

You are a senior full-stack engineer, AI systems architect, and production DevOps engineer building Co-Op — an open-source, self-hosted Enterprise AI Operating System. You have deep expertise in Python, TypeScript, FastAPI, Next.js, LangGraph, Temporal, Qdrant, LiteLLM, Docker, and Kubernetes.

REFERENCE DOCUMENT

You have been provided with: CoOp_Full_Engineering_Documentation_v4.0.docx

This document contains 18 complete engineering specifications including: Vision, SRS, System Architecture, Component Design, Backend Design, Frontend Design, Data Architecture, AI System Design, Control Flow, Implementation Plan, Roadmap, Repository Structure, API Design, Security Architecture, DevOps & Deployment, Observability Design, Developer Guidelines, and Testing Strategy.

READ THE ENTIRE DOCUMENT BEFORE WRITING A SINGLE LINE OF CODE. Understand every layer, every schema, every API contract, and every architectural decision before proceeding.

YOUR MISSION

Build the Co-Op Enterprise AI Operating System to production quality — exactly as specified in the engineering documentation. This is not a prototype or a demo. Every file you create must be production-grade: typed, tested, documented, and deployable.

PHASE 0 — START HERE (Week 1 Goal)

Build Phase 0 first and ONLY Phase 0. Do not build Phase 1, 2, or 3 infrastructure yet.

Phase 0 requires exactly 6 services running via Docker Compose:

1. Next.js 15 (App Router) — chat UI with SSE streaming and document upload
2. FastAPI — main API with /v1/chat/stream, /v1/documents, /v1/search, /v1/auth endpoints
3. LiteLLM → Ollama — Llama 3.2:3B on CPU, zero GPU required
4. Qdrant — vector database with HNSW, hybrid BM25+dense search
5. PostgreSQL 16 — users, conversations, documents, audit_events schema
6. Redis 7 — event bus (Streams) and cache

CORE CAPABILITIES TO DELIVER (Phase 0)

1. Document Upload: Accept PDF, DOCX, TXT, Markdown → parse with DeepDoc → semantic chunk → embed with all-MiniLM-L6-v2 → store in Qdrant
2. Hybrid Search: BM25 + dense vector (HNSW) combined via RRF → top-5 reranked results with citations
3. AI Chat: Streaming SSE responses → RAG-grounded answers with source citations (filename + page number)
4. Auth: JWT-based (no Keycloak yet) — API key or simple user/password login
5. Eval CI: DeepEval baseline test — Contextual Recall > 92%, Faithfulness > 90%

PRODUCTION QUALITY STANDARDS

Every file you write must meet these standards — no exceptions:

Python: Full type annotations, Pydantic v2 models for all schemas, async/await throughout, ruff-clean (zero warnings), 80%+ test coverage with pytest

TypeScript: Strict mode (no any), React functional components, TanStack Query for all data fetching, react-hook-form + zod for forms

Docker: All services have working health checks, restart: unless-stopped, no hardcoded secrets (use .env file)

API: Every endpoint has Pydantic request/response models, proper HTTP status codes, and OpenAPI docstring

Tests: Unit tests for all core functions, integration tests for all API endpoints, conftest.py with proper fixtures

REPOSITORY STRUCTURE

Use the exact monorepo structure from Doc 12 in the reference document. Do not invent your own structure. The root is a Turborepo monorepo with pnpm workspaces for TypeScript and Python uv workspaces for backend services.

DATABASE SCHEMA

Implement the exact PostgreSQL schema from Doc 5 in the reference document. Use SQLAlchemy 2.0 async ORM + Alembic for migrations. Use the Repository pattern for all database access — no raw SQL in routers.

RAG PIPELINE

Implement the exact hybrid search pipeline from Doc 8. Chunks: 512 tokens, 64 overlap. Embedding: all-MiniLM-L6-v2 (384-dim). Qdrant: HNSW (m=16, ef_construct=128). Search: dense + sparse BM25, merged via RRF (alpha=0.7). Always include source_file + page_number in every result.

AGENT ARCHITECTURE

For Phase 0, implement the Research Agent from Doc 8 as a LangGraph StateGraph. Implement the base AgentState TypedDict exactly as specified. The agent must: retrieve docs → rerank → generate answer → return with citations. No HITL needed in Phase 0 (read-only only).

API CONTRACTS

Implement EXACTLY the API endpoints from Doc 13. Use the exact request/response schemas shown. The /v1/chat/stream endpoint MUST use Server-Sent Events (SSE) and emit: {type:token}, {type:citation}, {type:done} events in that format.

FRONTEND CHAT UI

Build the Next.js chat interface as specified in Doc 6. Must have: streaming SSE token display, inline source citation cards (filename + page number), conversation history sidebar, and document upload page. Use Tailwind CSS + shadcn/ui. Use the useChat hook pattern from Doc 6.

CI/CD AND TESTING

Create the GitHub Actions CI workflow from Doc 15. It must run: ruff + mypy → pytest (80% coverage gate) → DeepEval RAG evaluation → Trivy security scan. All gates must pass before the workflow succeeds.

WHAT NOT TO BUILD YET

DO NOT build any of the following in Phase 0 — they belong to later phases:

❌Keycloak (Phase 1) — use simple JWT for now

❌Temporal workflows (Phase 1) — use direct function calls for now

❌Composio / MCP tools (Phase 2) — not needed yet

❌LLM Guard / NeMo Guardrails (Phase 1) — not needed yet

❌Istio / service mesh (Phase 2) — not needed yet
❌KubeRay / vLLM GPU (Phase 2) — run CPU-only Ollama for now
❌HITL approval flows (Phase 2) — not needed yet
❌Graphiti / Neo4j memory (Phase 2) — not needed yet

SUCCESS CRITERIA — DO NOT STOP UNTIL THESE ALL PASS

✅docker-compose up → all 6 services green within 5 minutes
✅Upload employee_handbook.pdf → status = indexed
✅Ask 'What is our remote work policy?' → cited answer with filename + page
✅P95 chat latency < 3 seconds on CPU-only inference
✅pytest → 80%+ coverage, all tests pass
✅DeepEval → Contextual Recall > 92%, Faithfulness > 90%
✅GitHub Actions CI → all 4 gates pass on every PR

EXECUTION ORDER

Follow this exact build order for Phase 0:

Step 1: Init monorepo structure (Turborepo + pnpm + Python uv)
Step 2: Create docker-compose.yml with all 6 services + health checks
Step 3: FastAPI skeleton — /health, /v1/auth, routing framework
Step 4: PostgreSQL schema + SQLAlchemy models + Alembic migrations
Step 5: Qdrant collection setup + embedding pipeline
Step 6: Document upload endpoint → DeepDoc parse → chunk → embed → store
Step 7: Hybrid search endpoint (BM25 + dense + RRF)
Step 8: LiteLLM config → Ollama model → chat stream endpoint
Step 9: LangGraph research agent (retrieve → rerank → generate)
Step 10: Next.js chat UI — streaming display + citation cards + doc upload
Step 11: Redis Streams event bus — document_ingestion queue
Step 12: DeepEval tests + GitHub Actions CI pipeline
Step 13: README.md — one-command deploy guide

IMPORTANT RULES

1. Never skip the reference document. Read all 18 specs before coding.
2. Never hallucinate APIs. Use only the endpoints and schemas documented.
3. Never hardcode secrets. All config via environment variables in .env.
4. Never write untested code. Every module needs a test file.
5. Never over-engineer Phase 0. If it is not in the Phase 0 spec, do not build it.
6. Always confirm before destructive changes. Ask before dropping tables or resetting stores.
7. Commit after every working step. Each of the 13 steps above should be its own git commit.

BEGIN NOW with Step 1: Initialize the monorepo structure.

When Phase 0 is complete and all 7 success criteria pass, report back and await instruction to begin Phase 1.

—— END OF PROMPT ——————————————————————————————————

Pro Tips for Getting the Best Results

Cursor / Windsurf: Attach the engineering doc, open composer, paste the prompt, select 'Apply to all files'.

Claude Projects: Upload the engineering doc to the Project, then paste the prompt in the conversation.

GitHub Copilot: Use Copilot Chat with @workspace, paste the prompt, reference the doc via #file.

If the agent loses context mid-build: paste just the EXECUTION ORDER section and the current step.

After each step is complete: run docker-compose up and verify the health check before continuing.

PART 2 — Complete Task Registry

Every task required to build Co-Op from Phase 0 to Phase 4. Each task has an ID, estimated time, dependencies, step-by-step instructions, and a clear acceptance criterion. Work through them in order within each phase.

Phase	Tasks	Duration	Services	Goal
Phase 0	T-00 to T-09	1–2 weeks	6	Company GPT — chat with company docs
Phase 1	T-10 to T-19	Month 2	12	Enterprise Search — SSO, multi-modal RAG, Temporal
Phase 2	T-20 to T-31	Month 3–5	20	Agent Automation — HITL, Composio, Kubernetes
Phase 3	T-32 to T-41	Month 6–8	25	Enterprise OS — graph memory, Vault, Istio
Phase 4	T-42 to T-51	Month 9–12	30	AI Platform — KubeRay, multi-cluster, marketplace

Dependency Rules

All Phase 0 tasks (T-00 to T-09) must be complete before starting Phase 1.
Within a phase, tasks with listed dependencies must be completed first.
The acceptance criterion for each task must PASS — not just 'code written' but verified working.
Every task should result in at least one git commit with a meaningful message.

PHASE 0 — Company GPT — 6 Services, Docker Compose

Week 1–2

T-00 Initialize Monorepo & Development Environment [Phase 0] CRITICAL

Est. Time: 4 hours **Depends On:** None

Steps to Complete:

1. Install Node.js 20+, Python 3.12+, Docker Desktop, pnpm 9+, uv (Python package manager)
2. Create root directory: mkdir co-op && cd co-op
3. Initialize Turborepo: npx create-turbo@latest . --package-manager pnpm
4. Create Python workspace: touch pyproject.toml with uv workspace config
5. Create directory structure exactly as in Doc 12: apps/, services/, agents/, infrastructure/, evaluation/, docs/
6. Initialize git: git init, create .gitignore (node_modules, .env, __pycache__, .venv, *.pyc)
7. Create .env.example with all required variables (DB_PASS, REDIS_URL, QDRANT_URL, LITELLM_URL, SECRET_KEY)
8. Create turbo.json pipeline: build, test, lint tasks
9. Push initial commit: git commit -m 'chore: initialize co-op monorepo'

✅**Done When:** pnpm install runs without errors. Directory structure matches Doc 12 exactly. git log shows first commit.

T-01 Create Docker Compose with 6 Services [Phase 0] CRITICAL

Est. Time: 3 hours **Depends On:** T-00

Steps to Complete:

1. Create infrastructure/docker/docker-compose.yml
2. Add postgres:16-alpine service: POSTGRES_DB=coop, POSTGRES_USER=coop, POSTGRES_PASSWORD=${DB_PASS}, named volume postgres_data
3. Add redis:7-alpine service: --appendonly yes, named volume redis_data
4. Add qdrant/qdrant:v1.9.0 service: port 6333, named volume qdrant_data
5. Add ollama/ollama service: port 11434, named volume ollama_data, entrypoint to pull llama3.2:3b
6. Add LiteLLM service: ghcr.io/berriai/litellm, mount litellm_config.yaml, port 4000, depends_on ollama
7. Add co-op-api service: build ./services/api, port 8000, all env vars from .env, depends_on postgres + redis + qdrant + inference, healthcheck: curl /health
8. Add co-op-web service: build ./apps/web, port 3000, NEXT_PUBLIC_API_URL=http://localhost:8000
9. Create infrastructure/docker/litellm_config.yaml: model_list with ollama/llama3.2:3b on http://ollama:11434
10. Test: cp .env.example .env && echo DB_PASS=testpass123 >> .env && docker-compose config (no errors)

✅**Done When:** docker-compose config runs without YAML errors. All 8 services defined with correct dependencies and health checks.

T-02 FastAPI Service Skeleton [Phase 0] CRITICAL

Est. Time: 4 hours **Depends On:** T-01

Steps to Complete:

1. Create services/api/ with pyproject.toml (fastapi, uvicorn, sqlalchemy, asyncpg, pydantic, redis, qdrant-client, httpx)
2. Create services/api/main.py: FastAPI app factory with lifespan context manager, CORS middleware, route includes
3. Create services/api/config.py: pydantic-settings BaseSettings class reading DATABASE_URL, REDIS_URL, QDRANT_URL, LITELLM_URL, SECRET_KEY from environment
4. Create services/api/dependencies.py: get_db() async session factory, get_redis() connection pool, verify_token() JWT dependency
5. Create services/api/routers/__init__.py and empty router files: auth.py, chat.py, documents.py, search.py, health.py
6. Implement GET /health endpoint returning {status: ok, version: 0.1.0, services: {db: ok, redis: ok, qdrant: ok}}
7. Create services/api/Dockerfile: python:3.12-slim, install uv, copy pyproject.toml, uv sync, CMD uvicorn main:app --host 0.0.0.0 --port 8000
8. Write services/api/tests/conftest.py with pytest fixtures: test_app (TestClient), test_db (async SQLAlchemy), mock_redis
9. Write test_health.py: assert GET /health returns 200 and correct schema

✅**Done When:** docker-compose up co-op-api starts. curl http://localhost:8000/health returns {status: ok}. pytest tests/test_health.py passes.

T-03 PostgreSQL Schema & Alembic Migrations [Phase 0] CRITICAL

Est. Time: 5 hours **Depends On:** T-02

Steps to Complete:

1. Create services/api/db/session.py: async SQLAlchemy engine with asyncpg, session factory, Base declarative
2. Create services/api/db/models/ directory with models for: User, Tenant, Document, DocumentChunk, Conversation, Message, AuditEvent, CostEvent — exactly as in Doc 5 schema
3. Add all fields from Doc 5 spec: UUID primary keys, tenant_id foreign keys, JSONB fields, TIMESTAMPTZ timestamps
4. Initialize Alembic: cd services/api && alembic init db/migrations
5. Configure alembic.ini and env.py to use async SQLAlchemy engine from config
6. Generate initial migration: alembic revision --autogenerate -m 'initial schema'
7. Review generated migration — verify all tables, columns, and indexes match Doc 5
8. Create services/api/db/repositories/ with BaseRepository class and UserRepository, DocumentRepository, AuditRepository
9. Write tests: test_repositories.py — create/read/update user, document CRUD, audit event append

✅**Done When:** alembic upgrade head runs without errors. All tables exist in PostgreSQL. Repository tests pass with test database.

T-04 JWT Authentication [Phase 0] HIGH

Est. Time: 3 hours **Depends On:** T-03

Steps to Complete:

1. Install python-jose[cryptography] and passlib[bcrypt] dependencies
2. Create services/api/routers/auth.py with POST /v1/auth/token (email+password → JWT), POST /v1/auth/refresh, GET /v1/auth/me
3. Create access token: RS256 signed JWT, exp=15min, payload: {sub: user_id, tenant_id, role, email}
4. Create refresh token: HS256, exp=7 days, stored in DB (or Redis with TTL)
5. Create verify_token() FastAPI dependency: decode JWT, check expiry, return UserContext(user_id, tenant_id, role)

6. Seed a default admin user on startup (email: admin@coop.local, password: from ADMIN_PASSWORD env var)
7. Write tests: test_auth.py — login success, login wrong password (401), token refresh, expired token (401), missing token (401)
8. Test manually: curl -X POST /v1/auth/token -d email=admin@coop.local -d password=... → returns {access_token, refresh_token}

✅**Done When:** All auth tests pass. JWT tokens decode correctly. Protected endpoints return 401 without valid token.

T-05 Qdrant Setup & Embedding Pipeline [Phase 0] CRITICAL

Est. Time: 5 hours **Depends On:** T-02

Steps to Complete:

1. Add sentence-transformers to pyproject.toml (all-MiniLM-L6-v2 model)
2. Create services/api/rag/embedder.py: load all-MiniLM-L6-v2 on startup, encode() method returning float32[384]
3. Create services/api/rag/qdrant_client.py: async Qdrant client wrapper, create_collection() creating co_op_documents with dense (384-dim, Cosine) + sparse (BM25) vectors and HNSW config (m=16, ef_construct=128)
4. Create services/api/rag/chunker.py: semantic_chunk(text, chunk_size=512, overlap=64) → list of chunks with metadata
5. Create bm25_encoder.py: encode text to sparse vector format Qdrant accepts
6. Create services/api/rag/indexer.py: upsert_chunks(chunks, document_id, tenant_id) → batch upsert to Qdrant with all payload fields from Doc 7
7. On startup: call create_collection() (idempotent — skip if exists)
8. Write tests: test_embedder.py (output shape [384]), test_chunker.py (chunk sizes), test_indexer.py (mock Qdrant upsert)

✅**Done When:** Qdrant collection co_op_documents created with correct vector config. Embedding model loads. Chunker produces correct chunk sizes with overlap. Unit tests pass.

T-06 Document Upload & Ingestion Pipeline [Phase 0] CRITICAL

Est. Time: 6 hours **Depends On:** T-03 T-05

Steps to Complete:

1. Add ragflow-sdk or deepdoc to dependencies (or use PyMuPDF + python-docx for Phase 0 parsing)
2. Create services/api/rag/parser.py: parse_file(path, file_type) → str (Markdown text). Support: PDF (PyMuPDF), DOCX (python-docx), TXT, MD
3. Create POST /v1/documents endpoint in documents.py: accept multipart file upload, validate type (pdf/docx/txt/md) and size (<50MB), save to local volume (MinIO optional in Phase 0), insert document record (status=pending), publish to Redis Streams document_ingestion_queue, return {id, filename, status}
4. Create services/api/events/consumer.py: Redis Streams XREADGROUP consumer reading document_ingestion_queue
5. Create ingestion worker (background task or separate process): read event → parse → chunk → embed → upsert Qdrant → update document status=indexed
6. Create GET /v1/documents/{id}/status endpoint returning current status and chunk_count
7. Write end-to-end test: upload test.pdf → poll status → assert status=indexed, chunk_count>0
8. Write integration test: upload + search — verify indexed content is retrievable

✅**Done When:** POST /v1/documents accepts PDF upload. Status transitions pending → processing → indexed. Document chunks appear in Qdrant. Integration test passes.

T-07 Hybrid Search & RAG Retrieval [Phase 0] CRITICAL

Est. Time: 5 hours **Depends On:** T-05 T-06

Steps to Complete:

1. Create services/api/rag/retriever.py: async hybrid_search(query, tenant_id, top_k=10, alpha=0.7) function
2. Implement dense search: embed query → Qdrant search with tenant_id payload filter → top 20 results
3. Implement sparse BM25 search: bm25_encode(query) → Qdrant search with same filter → top 20 results
4. Implement RRF merge: reciprocal_rank_fusion(dense, sparse, alpha) → merged top_k list
5. Create reranker.py: simple cross-encoder rerank (use sentence-transformers cross-encoder/ms-marco-MiniLM-L-6-v2) → top 5
6. Create POST /v1/search endpoint: {query, top_k, filters} → [{content, score, source, page, document_id}]
7. Add latency tracking: log search time, emit to metrics
8. Write tests: test_retriever.py — search returns results in correct format, tenant isolation (tenant A cannot see tenant B docs), alpha weighting works

✅**Done When:** POST /v1/search returns hybrid results with source citations in < 200ms on 1000 vectors. Tenant isolation verified. Tests pass.

T-08 LangGraph Research Agent & Chat Stream Endpoint [Phase 0] CRITICAL

Est. Time: 6 hours **Depends On:** T-07

Steps to Complete:

1. Create agents/research_agent/state.py: AgentState TypedDict exactly as in Doc 8 (messages, documents, tool_calls, tenant_id, user_id, cost_usd, iteration)
2. Create agents/research_agent/nodes.py: async retrieve_docs(state) calling hybrid_search, async rerank_results(state) calling reranker, async generate_answer(state) calling LiteLLM
3. Create agents/research_agent/graph.py: StateGraph with START→retrieve_docs→rerank_results→generate_answer→END
4. Compile graph with PostgreSQL checkpointer (AsyncPostgresSaver) for state persistence
5. Create services/api/rag/prompt_builder.py: build_rag_prompt(query, chunks) with SYSTEM_PROMPT from Doc 8, context block with citations
6. Create POST /v1/chat/stream endpoint: validate auth, create/resume conversation, run agent graph, stream response via SSE
7. SSE events must emit exactly: data: {type:token, content:str}, data: {type:citation, source:str, page:int}, data: {type:done, conversation_id:str, cost_usd:float}
8. Save messages to PostgreSQL, write cost_event to cost_events table
9. Write tests: test_chat.py — SSE stream returns tokens, citations present in response, cost event written to DB

✅**Done When:** POST /v1/chat/stream streams tokens in SSE format. Response includes inline citations. Conversation saved to DB. Cost event written. All tests pass.

T-09 Next.js Chat UI & Document Upload [Phase 0] CRITICAL

Est. Time: 8 hours **Depends On:** T-08

Steps to Complete:

1. Initialize apps/web: npx create-next-app@latest with TypeScript, Tailwind, App Router
2. Install dependencies: shadcn/ui, @tanstack/react-query, zustand, react-hook-form, zod, lucide-react

3. Create apps/web/lib/api.ts: typed fetch wrappers for all Phase 0 API endpoints
4. Create apps/web/store/chatStore.ts: Zustand store for active conversation, messages array
5. Create apps/web/hooks/useChat.ts: SSE streaming hook from Doc 6 (reads token/citation/done events)
6. Build app/(app)/chat/page.tsx: left sidebar (conversation list), main chat area, message input
7. Build components/chat/MessageBubble.tsx: renders markdown + inline CitationCard component
8. Build components/chat/CitationCard.tsx: shows source filename + page number, links to doc
9. Build components/chat/StreamingText.tsx: animates token-by-token text appearance
10. Build app/(app)/docs/page.tsx: file drop zone (react-dropzone), upload progress, document list with status badges
11. Add OpenTelemetry basic setup (console exporter for Phase 0)
12. Write Playwright E2E: test_chat.spec.ts — upload PDF, send query, verify citation appears in response

✅**Done When:** docker-compose up → http://localhost:3000 shows chat UI. Upload a PDF. Ask a question. Streaming response with citation card appears. E2E test passes.

T-09B DeepEval CI Pipeline & README [Phase 0] HIGH

Est. Time: 4 hours **Depends On:** T-09

Steps to Complete:

1. Create evaluation/deepeval/datasets/ — seed with 50 Q&A pairs from a sample company PDF
2. Create evaluation/deepeval/test_rag.py with ContextualRecallMetric(0.92) and FaithfulnessMetric(0.90) tests
3. Create .github/workflows/ci.yml: trigger on PR, jobs: lint (ruff + mypy + eslint), test (pytest --cov 80%), ai_eval (deepeval test run), security (trivy)
4. Set DEEPEVAL CI environment variable so DeepEval uses API key from GitHub Secrets
5. Create README.md at repo root: title, badges (CI status, license), one-command deploy section, 5-minute quickstart, architecture diagram (text)
6. Quickstart section: git clone → cp .env.example .env → fill in DB_PASS → docker-compose up → open localhost:3000
7. Push to GitHub, open a test PR, verify all CI checks pass
8. Tag release v0.1.0-phase0

✅**Done When:** GitHub Actions CI shows all green checks. README deploys in one copy-paste. DeepEval recall > 92% on sample dataset. v0.1.0 tag created.

PHASE 1 Month 2 — Enterprise Knowledge Platform — 12 Services, Docker Swarm

T-10 Keycloak Enterprise Authentication [Phase 1] CRITICAL

Est. Time: 5 hours **Depends On:** All Phase 0 tasks

Steps to Complete:

1. Add keycloak service to docker-compose.prod.yml: quay.io/keycloak/keycloak:24, realm config, admin credentials from env
2. Create Keycloak realm 'coop' with OIDC client 'coop-api', configure redirect URIs for Next.js
3. Configure user federation (LDAP optional) and groups mapping to Co-Op RBAC roles (admin, manager, agent_operator, analyst, viewer)
4. Replace simple JWT in FastAPI with Keycloak JWKS validation: fetch public keys from /realms/coop/protocol/openid-connect/certs
5. Update verify_token() dependency to decode Keycloak JWT and extract roles from realm_access.roles claim
6. Update Next.js auth flow: use next-auth with Keycloak provider, store tokens in HttpOnly cookie (not localStorage)
7. Add SAML 2.0 configuration for Azure AD / Okta enterprise SSO federation
8. Write tests: test_keycloak_auth.py — token from Keycloak validates in FastAPI, role-based access works

✓**Done When:** Login via Keycloak works. Admin role can access /admin endpoints. Viewer role gets 403 on admin endpoints. Enterprise SSO config documented.

T-11 LLM Guard Prompt Security Middleware [Phase 1] HIGH

Est. Time: 4 hours **Depends On:** T-10

Steps to Complete:

1. Add llm-guard to services/api/pyproject.toml
2. Create services/guardrails/scanner.py: configure input scanners (PromptInjection, Sensitive, Toxicity) and output scanners (Sensitive, Deanonymize)
3. Create FastAPI middleware class LLMGuardMiddleware: intercept POST /v1/chat/stream requests, run input scanner, block if injection detected (return 400 with reason), run output scanner on generated text before streaming
4. Add PII anonymization: replace SSNs, credit card numbers, phone numbers in both input and output
5. Add audit logging: every blocked request writes to audit_events with event_type=prompt_injection_blocked
6. Create Prometheus counter: llm_guard_blocked_requests_total{scanner_type}
7. Write tests: test_guardrails.py — injection attempt returns 400, PII in query gets anonymized, legitimate query passes

✓**Done When:** Prompt injection attempt returns 400. PII in prompts is anonymized before reaching LLM. Blocked requests appear in audit log. Prometheus counter increments.

T-12 Traefik API Gateway with TLS [Phase 1] HIGH

Est. Time: 3 hours **Depends On:** T-10

Steps to Complete:

1. Add traefik:v3 service to docker-compose.prod.yml: ports 80, 443, 8080 (dashboard), mount traefik.yml config
2. Create infrastructure/docker/traefik.yml: entryPoints (web→443, websecure→443), certificatesResolvers (Let's Encrypt ACME or self-signed for local dev)
3. Add rate limiting middleware: 100 req/min per IP for /v1/chat, 1000 req/min for /v1/search
4. Add HTTP→HTTPS redirect middleware
5. Label co-op-api service for Traefik routing: traefik.enable=true, router rule Host(api.yourdomain.com), TLS certresolver
6. Add health check dashboard accessible at https://traefik.yourdomain.com:8080 (IP-restricted)
7. Write test: verify HTTP redirects to HTTPS, rate limit returns 429 after threshold

✅**Done When:** API accessible via HTTPS. HTTP redirects to HTTPS. Rate limiting returns 429 on 101st request. Dashboard accessible.

T-13 RAGFlow Full Integration with DeepDoc [Phase 1] HIGH

Est. Time: 6 hours **Depends On:** T-06

Steps to Complete:

1. Replace simple PyMuPDF parser with RAGFlow + DeepDoc for production-quality document parsing
2. Install ragflow-sdk or run RAGFlow as a separate Docker service (ragflow/ragflow image)
3. Update services/api/rag/parser.py: route PDF files through DeepDoc VLM parser which preserves table structure, headers, and multi-column layouts
4. Test complex documents: Excel spreadsheets (table structure preserved), PowerPoint (slide text extracted), scanned PDFs (OCR via DeepDoc)
5. Implement document preview endpoint: GET /v1/documents/{id}/preview → returns structured markdown with table markup
6. Update chunker to respect document structure: do not split mid-table or mid-header
7. Run DeepEval benchmark on complex test set: verify recall stays > 92% on tables and structured content
8. Update ingestion tests: add complex PDF with tables as test fixture

✅**Done When:** RAGFlow parses complex PDFs correctly. Table content is retrievable via search. DeepEval recall > 92% on complex document test set.

T-14 Temporal Workflow Engine Integration [Phase 1] HIGH

Est. Time: 6 hours **Depends On:** T-06

Steps to Complete:

1. Add temporalio/server service to docker-compose.prod.yml (Temporal server + PostgreSQL backend)
2. Add temporalio SDK to services/api/pyproject.toml
3. Create services/ingestion/ as a separate Python service with its own Dockerfile
4. Convert document ingestion pipeline to Temporal workflow: create DocumentIngestionWorkflow with 5 activities (parse, chunk, embed, index, update_status) each with retry policy (max 3, backoff 2x, timeout 120s)
5. Create Temporal worker in services/ingestion/worker.py: register workflow and activities, connect to Temporal server
6. Update FastAPI documents router: after file save, use Temporal client to start_workflow(DocumentIngestionWorkflow) instead of Redis publish
7. Test failure recovery: kill worker mid-ingestion, restart it — verify workflow resumes from last successful activity
8. Write integration test: upload PDF, kill worker, restart worker, verify document ends up indexed

✅**Done When:** Document ingestion runs as Temporal workflow. Killing the worker mid-process and restarting it results in successful completion (no data loss). Temporal UI shows workflow history.

T-15 Docker Swarm Production Deployment [Phase 1] HIGH

Est. Time: 4 hours **Depends On:** T-10 T-11 T-12 T-13 T-14

Steps to Complete:

1. Create infrastructure/docker/docker-compose.swarm.yml with deploy sections (replicas, update_config, restart_policy)
2. Configure secrets in Swarm: docker secret create db_password — reference in services instead of env vars
3. Set resource limits: api service (0.5 CPU, 512M RAM), qdrant (2 CPU, 4G RAM), postgres (1 CPU, 2G RAM)
4. Add rolling update config: parallelism=1, delay=10s, failure_action=rollback
5. Create 3-node Swarm setup script: scripts/swarm_init.sh — docker swarm init + worker join tokens
6. Deploy: docker stack deploy -c docker-compose.swarm.yml coop
7. Test: docker service ls — all services Running. docker service ps coop_api — no failures
8. Run Locust load test: 500 concurrent users, verify P95 < 3s, error rate < 0.1%

✅**Done When:** docker stack deploy succeeds. All 12 services running across 3 nodes. Locust 500-user test: P95 < 3s, error rate < 0.1%. Rolling update without downtime demonstrated.

T-16 Prometheus + Grafana Observability Stack [Phase 1] MEDIUM

Est. Time: 5 hours **Depends On:** T-15

Steps to Complete:

1. Add prometheus, grafana, loki, promtail services to docker-compose.prod.yml
2. Create infrastructure/docker/prometheus.yml: scrape configs for all 12 services, 15s scrape interval
3. Add Prometheus metrics to FastAPI: from prometheus_fastapi_instrumentator import Instrumentator — instrument all routes
4. Add custom business metrics: co_op_rag_recall_gauge, co_op_agent_success_rate, co_op_cost_per_query_histogram
5. Create Grafana datasources: Prometheus + Loki
6. Import/create Grafana dashboard with the 8 golden signals from Doc 16: P95 latency, error rate, RAG recall, agent success, GPU util (N/A Phase 1), queue depth, cost/hour, HITL pending
7. Configure Grafana alerts: P95 > 3s for 5min → Slack webhook notification
8. Create Loki log aggregation: all services log JSON to stdout, promtail scrapes and ships to Loki

✅**Done When:** Grafana dashboard shows all 8 golden signals. Prometheus scrapes all services. Loki has logs from all containers. Slack alert fires on simulated latency spike.

T-17 Admin Panel & Document Library UI [Phase 1] MEDIUM

Est. Time: 6 hours **Depends On:** T-10 T-13

Steps to Complete:

1. Build app/(app)/admin/ pages: users/page.tsx (user list + create/edit modal), integrations/page.tsx (placeholder), settings/page.tsx
2. Build app/(app)/docs/page.tsx: full document library with columns (name, type, status, chunk count, upload date, uploaded by), delete action
3. Add document preview modal: GET /v1/documents/{id}/preview → show structured markdown
4. Add search with filters to knowledge search page: filter by file_type, date_after, date_before, tag

5. Build analytics/page.tsx: basic charts using Recharts — queries per day, cost per day, top documents queried
6. Add role-based UI: admin users see /admin nav item, viewers do not. Protect routes with useAuth() hook
7. Write Playwright tests: test_admin.spec.ts — admin can create user, viewer cannot access /admin (redirects)

✅**Done When:** Admin panel accessible to admin role only. Document library shows all uploaded docs with correct status. Analytics shows real data from cost_events table. Playwright tests pass.

T-18 Hybrid BM25 + Dense Search Upgrade [Phase 1] HIGH

Est. Time: 3 hours **Depends On:** T-07 T-13

Steps to Complete:

1. Upgrade Qdrant collection to enable sparse vectors alongside dense (recreate collection if needed)
2. Update bm25_encoder.py: use rank_bm25 library with vocabulary built from all indexed documents
3. Update hybrid_search() to perform true parallel sparse + dense queries to Qdrant
4. Tune RRF fusion: test alpha values 0.5, 0.7, 0.9 on benchmark dataset — pick highest recall
5. Add reranking with cross-encoder model: sentence-transformers/cross-encoder/ms-marco-MiniLM-L-6-v2
6. Run DeepEval benchmark: verify Contextual Recall improves to > 95% with hybrid + reranking vs. 92% dense-only baseline
7. Log search latency breakdown (dense ms, sparse ms, merge ms, rerank ms) to Prometheus

✅**Done When:** Hybrid search recall > 95% on 1000-document benchmark. Latency breakdown visible in Prometheus. BM25 + reranker working together.

T-19 Phase 1 Review & Docker Swarm Validation [Phase 1] CRITICAL

Est. Time: 2 hours **Depends On:** T-10 through T-18

Steps to Complete:

1. Run full test suite: pytest services/api/tests/ — all tests pass, coverage > 80%
2. Run Playwright E2E suite: all scenarios pass on 3-node Swarm
3. Run DeepEval: Recall > 95%, Faithfulness > 90%, Hallucination < 3%
4. Run Locust: 500 concurrent users, 30 minutes, P95 < 3s, error rate < 0.1%
5. Run Promptfoo basic set: 100 adversarial prompts, zero successful injections
6. Verify all Grafana dashboards populated with real metrics
7. Document Phase 1 in CHANGELOG.md, tag release v0.2.0-phase1
8. Demo: record 5-minute video showing doc upload, SSO login, complex PDF Q&A, admin panel

✅**Done When:** All CI gates pass. Locust 500-user test passes. Grafana dashboards live. v0.2.0 tagged. Demo video recorded.

PHASE 2 — Agent Automation — 20 Services, Kubernetes

Month 3–5

T-20 Kubernetes Infrastructure Setup (EKS/GKE/AKS) [Phase 2] CRITICAL

Est. Time: 8 hours **Depends On:** All Phase 1 tasks

Steps to Complete:

1. Create infrastructure/terraform/ with modules for your target cloud (EKS for AWS, GKE for GCP, or AKS for Azure)
2. Provision: managed K8s cluster (3 nodes: 2x app nodes + 1x GPU-ready node), managed PostgreSQL, managed Redis (ElastiCache/Cloud Memorystore), VPC with private subnets
3. Install required K8s add-ons: cert-manager (TLS), external-dns (DNS automation), metrics-server (HPA), KEDA (event-driven autoscaling)
4. Create infrastructure/helm/co-op-core/ Helm chart: api Deployment, ingestion Deployment, inference Deployment, ConfigMap, Secrets (use external-secrets or Vault Phase 3), Services, Ingress
5. Configure HPA for co-op-api: CPU target 70%, min 2, max 10 replicas
6. Configure KEDA ScaledObject for co-op-worker: Redis Streams queue depth trigger, min 1, max 20 workers
7. Deploy with: helm install co-op ./infrastructure/helm/co-op-core -n co-op --create-namespace
8. Test: kubectl get pods -n co-op — all Running. kubectl top pods — resource usage visible

✓**Done When:** terraform apply creates cluster. helm install deploys all pods. kubectl get pods shows all Running. HPA scales on load test.

T-21 LangGraph Agents with Full HITL [Phase 2] CRITICAL

Est. Time: 8 hours **Depends On:** T-20

Steps to Complete:

1. Upgrade research_agent to full HITL pattern from Doc 8: add conditional routing — if action is mutating, route to propose_action node before END
2. Implement interrupt_before=['propose_action'] in graph compile — graph pauses here
3. When interrupt fires: serialize proposed action payload + reasoning trace to hitl_approvals table (status=pending), publish to Redis Streams hitl_approvals
4. Create POST /v1/approvals/{id} endpoint: validate manager role, update hitl_approvals status=approved/rejected, publish Temporal signal to resume workflow
5. Create GET /v1/approvals endpoint: return pending approvals with evidence_pack JSON for the HITL inbox UI
6. Build app/(app)/approvals/page.tsx: approval card list, evidence pack display (reasoning trace summary), 1-click approve/reject buttons with confirmation dialog
7. Build components/approvals/EvidencePack.tsx: show agent reasoning steps, proposed action parameters, risk assessment
8. Add PostgreSQL checkpointer to LangGraph for state persistence across interrupt/resume cycles
9. Write tests: test_hitl.py — agent pauses on mutating action, approval resumes workflow, rejection cancels workflow

✓**Done When:** Agent proposes email-send action. Approval inbox shows request with evidence pack. Manager approves. Workflow resumes and executes tool. Rejection cancels workflow. All tests pass.

T-22 Composio MCP Gateway Integration [Phase 2] HIGH

Est. Time: 6 hours **Depends On:** T-21

Steps to Complete:

1. Create services/mcp_gateway/ FastAPI service with Composio SDK
2. Configure Composio: create account, get API key, set up apps (Slack, Gmail, GitHub, Jira, Notion)
3. Create services/mcp_gateway/routers/tools.py: POST /v1/tools/execute — receives tool name + parameters, validates against RBAC (agent cannot execute tools above its permission level), calls Composio action, returns result
4. Update LangGraph agent tools: replace placeholder tool functions with real MCP calls via HTTP to mcp_gateway service
5. Implement OAuth token management: Composio handles token refresh automatically — implement per-tenant OAuth connection storage in PostgreSQL
6. Add tool rate limiting: max 10 tool calls per agent run, max 100 tool calls per tenant per day
7. Write tests: test_mcp_tools.py — mock Composio, test Slack send, Gmail send, GitHub create PR, Jira create ticket
8. Write integration test: research agent finds answer → proposes Slack message → manager approves → message sent via Composio → confirmation in response

✅**Done When:** Agent executes real Slack message after HITL approval. Gmail, GitHub, and Jira tools working. Tool rate limiting enforced. OAuth tokens persisted per tenant.

T-23 React Flow Workflow Builder [Phase 2] HIGH

Est. Time: 8 hours **Depends On:** T-21

Steps to Complete:

1. Install @xyflow/react (React Flow v12) in apps/web
2. Create app/(app)/workflows/page.tsx: workflow library with create/edit/delete and run history
3. Create app/(app)/workflows/[id]/page.tsx: full-screen workflow canvas
4. Build components/workflow/WorkflowCanvas.tsx: React Flow wrapper with custom node types
5. Build custom node components: AgentNode (LangGraph agent), ToolNode (MCP tool), HITLNode (approval checkpoint), TriggerNode (schedule/webhook), ConditionNode (branching logic)
6. Implement workflow save: serialize React Flow graph (nodes + edges) to JSON, POST /v1/workflows
7. Implement workflow run: POST /v1/workflows/{id}/run → starts Temporal workflow, returns run_id
8. Implement live run status: WebSocket subscription to workflow run events → update node colors (running=yellow, done=green, error=red)
9. Add 5 starter workflow templates: Doc Summary → Slack, Support Ticket → Jira, PR Review → GitHub Comment, Weekly Report → Email, Policy Q&A

✅**Done When:** Workflow canvas renders. Nodes draggable and connectable. Workflow saves and loads. Run shows live status updates on nodes. 5 templates importable.

T-24 NeMo Guardrails Agent Safety [Phase 2] HIGH

Est. Time: 4 hours **Depends On:** T-22

Steps to Complete:

1. Install nemoguardrails in services/guardrails/
2. Create colang policy files: blocks.co (prevent overriding system prompt), tool_abuse.co (prevent calling financial tools without HITL), data_exfil.co (prevent outputting bulk data dumps)
3. Integrate NeMo runtime as a wrapper around LangGraph agent execution: every agent input/output passes through NeMo rails
4. Add guardrail violation logging to audit_events with event_type=guardrail_violation
5. Create Prometheus counter: co_op_guardrail_violations_total{policy_name}

6. Write Promptfoo tests specifically for agent safety: tool poisoning attempt, system prompt override, unauthorized financial action — all must fail
7. Test: attempt to instruct agent to call payment tool without HITL — guardrail must block it

✅**Done When:** Tool poisoning attempt blocked by NeMo policy. System prompt override rejected. Violation logged in audit_events. Promptfoo agent safety tests all pass.

T-25 vLLM GPU Inference (Optional for Phase 2) [Phase 2] MEDIUM

Est. Time: 4 hours **Depends On:** T-20

Steps to Complete:

1. Only proceed if GPU hardware or cloud GPU instance available (A10, A100, or RTX 3090+)
2. Add vllm Docker image to Kubernetes: deploy as GPU-specific workload with nodeSelector for GPU nodes
3. Update LiteLLM config to add vLLM as provider: model_list entry with api_base pointing to vLLM service
4. Implement semantic cost routing in LiteLLM config: simple queries → ollama/llama3.2:3b, complex multi-doc → vllm/mixtral-8x7b
5. Add GPU metrics to Prometheus: vLLM exposes /metrics with gpu_utilization, pending_requests, token_throughput
6. Test routing: verify simple 'What is our PTO policy?' goes to CPU model, complex analysis goes to GPU model
7. Benchmark: measure token throughput improvement vs Ollama CPU

✅**Done When:** vLLM serving Mixtral 8x7B on GPU. LiteLLM routes complex queries to GPU, simple queries to CPU. GPU metrics in Prometheus. 10x+ throughput improvement measured.

T-26 Canary Deployment Pipeline [Phase 2] HIGH

Est. Time: 4 hours **Depends On:** T-20

Steps to Complete:

1. Update .github/workflows/deploy.yml: after CI passes on main, build Docker image, push to registry with ${GITHUB_SHA} tag
2. Helm upgrade with canary: --set image.tag=${SHA} --set canary.enabled=true --set canary.weight=5
3. Add monitoring step: sleep 900 (15 min), query Prometheus for error rate on canary pods
4. If error_rate < 1%: helm upgrade with canary.weight=100. If error_rate >= 1%: helm rollback co-op
5. Add manual approval gate in GitHub Actions for production promotion (environment: production with required reviewers)
6. Test: deploy a deliberately broken version, verify auto-rollback triggers within 15 minutes
7. Add deployment notification: Slack webhook message on deploy start and deploy success/failure

✅**Done When:** Canary deploy creates 5% traffic split. Broken deploy auto-rolls back within 15 min. Successful deploy auto-promotes. Slack notification sent. Manual gate works.

T-27 Agent Monitor & Analytics Dashboard Upgrade [Phase 2] MEDIUM

Est. Time: 5 hours **Depends On:** T-23 T-16

Steps to Complete:

1. Build app/(app)/agents/page.tsx: agent run gallery with status filters (running/success/failed), sortable by cost/time
2. Build app/(app)/agents/[id]/page.tsx: full run detail page — LangGraph trace visualization (node sequence), tool calls timeline, cost breakdown, full message history

3. Embed Grafana panels via iframe (or rebuild in Recharts): cost per agent run, success rate trend, HITL approval rate
4. Build analytics cost dashboard: daily cost by model, cost by tenant, projected monthly cost, cost per query trend
5. Add agent run search: filter by agent_type, date range, user, cost > threshold
6. Create CSV export for analytics data (for finance team billing)
7. Write tests: test_analytics.py — cost aggregation queries return correct totals

✅**Done When:** Agent monitor shows live runs with LangGraph trace. Cost dashboard shows model breakdown. CSV export works. Analytics tests pass.

T-28 Full Promptfoo Red-Teaming CI Integration [Phase 2] HIGH

Est. Time: 3 hours **Depends On:** T-24

Steps to Complete:

1. Create evaluation/promptfoo/config.yaml with all test categories from Doc 18: prompt injection (100 attacks), jailbreak (100 variants), PII extraction (50 attempts), cross-tenant data access (50 attempts), agent tool abuse (50 attempts), system prompt extraction (50 attempts)
2. Add promptfoo job to .github/workflows/ci.yml: runs on every PR against staging environment
3. Set failure threshold: any successful attack = CI failure, PR cannot merge
4. Generate HTML report: promptfoo eval --output evaluation/promptfoo/reports/latest.html
5. Archive reports as GitHub Actions artifacts
6. Create weekly scheduled run: .github/workflows/scheduled_redteam.yml runs every Monday 2am
7. Test: verify a known injection attempt is caught. Verify CI fails when injection succeeds

✅**Done When:** 500+ adversarial tests run on every PR. Any successful attack blocks merge. HTML report generated. Weekly scheduled run configured. Known injection blocked in test.

T-29 Tauri Desktop Client [Phase 2] MEDIUM

Est. Time: 8 hours **Depends On:** T-23

Steps to Complete:

1. Initialize apps/desktop/: npx create-tauri-app with React + TypeScript template
2. Configure Tauri to load the co-op web app from https://app.yourdomain.com (production) or http://localhost:3000 (dev)
3. Implement native OS notifications: when HITL approval is requested, show OS notification via Tauri notification plugin
4. Implement local file watcher: watch ~/Documents/co-op-sync/ folder, auto-upload new/modified files via API
5. Add system tray icon: running status, quick query input (type question, get answer without opening full window)
6. Build offline mode: cache last 50 conversations in SQLite via Tauri store plugin
7. Package for macOS (.dmg), Windows (.msi), Linux (.AppImage)
8. Write tests: Tauri unit tests for file watcher logic, notification trigger

✅**Done When:** Desktop app installs and connects to API. OS notification fires on HITL request. File watcher auto-uploads new PDFs. System tray works. All 3 platform packages build successfully.

T-30 Phase 2 Kubernetes Load Testing & Validation [Phase 2] CRITICAL

Est. Time: 3 hours **Depends On:** T-20 through T-29

Steps to Complete:

1. Run Locust: 2000 concurrent users, 30 minutes, target P95 < 2s, error rate < 0.1%
2. Verify HPA scaled co-op-api during test (kubectl get hpa -n co-op shows scale-out events)
3. Verify KEDA scaled agent workers during test
4. Run full DeepEval suite: Recall > 95%, Faithfulness > 90%, Tool Correctness > 95%
5. Run Promptfoo full 500-test suite: zero successful attacks
6. Run Playwright E2E: all scenarios including HITL approval flow, workflow builder, agent monitor
7. Verify all Grafana dashboards populated correctly at 2000-user scale
8. Tag release v0.3.0-phase2, update CHANGELOG

✅**Done When:** Locust 2000-user test passes. HPA and KEDA auto-scaled. All AI eval gates pass. Zero security vulnerabilities. v0.3.0 released.

PHASE 3 — Enterprise OS — Graph Memory, Vault, Istio

Month 6–8

T-32 Graphiti Temporal Knowledge Graph Memory [Phase 3] HIGH

Est. Time: 6 hours **Depends On:** All Phase 2

Steps to Complete:

1. Add graphiti-core to services/api/pyproject.toml
2. Create services/api/memory/graphiti_client.py: connect to Neo4j backend (add neo4j service to Helm chart), initialize Graphiti with LiteLLM model
3. Implement agent memory integration: after each successful agent run, extract entity-relationship triples from conversation using Graphiti.add_episode()
4. Implement memory retrieval: before agent runs, call Graphiti.search(query) to retrieve relevant past facts, inject into system prompt
5. Implement temporal fact invalidation: when a fact changes (e.g., employee moves departments), Graphiti automatically invalidates the old edge and adds the new one with validity timestamps
6. Test: tell agent 'John is in Marketing' → later tell 'John moved to Engineering' → ask 'What team is John on?' → agent answers Engineering (not Marketing)

✅**Done When:** Agents remember facts across conversations. Contradictory fact updates resolve correctly. Temporal queries return historically accurate answers. Integration tests pass.

T-33 HashiCorp Vault Secrets Management [Phase 3] HIGH

Est. Time: 5 hours **Depends On:** All Phase 2

Steps to Complete:

1. Deploy Vault in Kubernetes: helm install vault hashicorp/vault with Raft HA storage (3 replicas)
2. Initialize and unseal Vault: vault operator init, store unseal keys in secure external location
3. Configure Kubernetes auth method: Vault authenticates service accounts via K8s JWT
4. Create database secrets engine: Vault dynamically generates PostgreSQL credentials with TTL 1hr, auto-rotated
5. Create KV secrets engine: store all API keys (Composio, external LLM providers) in Vault KV v2
6. Update all services: replace static env var secrets with Vault agent sidecar injection (annotations in pod spec)
7. Remove ALL secrets from docker-compose files and Kubernetes manifests — nothing hardcoded
8. Write audit test: verify that rotating DB credentials does not cause service downtime

✅**Done When:** Zero secrets in code, config files, or K8s manifests. Vault generates DB credentials dynamically. Credential rotation causes zero downtime. Vault audit log shows all secret access.

T-34 Istio Ambient Service Mesh [Phase 3] MEDIUM

Est. Time: 5 hours **Depends On:** T-33

Steps to Complete:

1. Install Istio Ambient mode in K8s cluster: istioctl install --set profile=ambient
2. Label co-op namespace for ambient: kubectl label namespace co-op istio.io/dataplane-mode=ambient
3. Configure mTLS PeerAuthentication: STRICT mode — all inter-service traffic must be mTLS encrypted

4. Create AuthorizationPolicy rules: co-op-api can call co-op-inference, co-op-worker can call co-op-api, external traffic only via ingress gateway
5. Configure traffic management: DestinationRule with connection pool limits and outlier detection for circuit breaking
6. Add Kiali visualization: helm install kiali — visualize service mesh topology and health
7. Verify in Kiali: all arrows between services show mTLS lock icons

✅**Done When:** All inter-service traffic encrypted with mTLS. Kiali shows mesh topology. AuthorizationPolicy blocks unauthorized service-to-service calls. Circuit breaker trips on simulated downstream failure.

T-35 Multi-Agent CrewAI Teams [Phase 3] MEDIUM

Est. Time: 6 hours **Depends On:** T-21

Steps to Complete:

1. Install crewai to services/api/pyproject.toml
2. Create agents/crew_templates/ with 3 pre-built crew examples: Research → Draft → Review crew, Support Ticket → Resolution → QA crew, Data Analysis → Report → Distribution crew
3. Implement crew execution via Temporal workflow: each crew member is a Temporal activity with retry logic
4. Add crew visualization in workflow builder: CrewNode shows member agents and their roles
5. Implement crew result aggregation: final output collects outputs from all agents, generates combined report
6. Write benchmark: crew vs. single agent on complex research task — measure output quality and latency

✅**Done When:** 3-agent research crew produces higher-quality report than single agent (measured by DeepEval). Crew runs as Temporal workflow with full retry protection. Crew nodes visible in workflow builder.

T-36 SOC2 Type II Audit Preparation [Phase 3] HIGH

Est. Time: 10 hours **Depends On:** T-33 T-34

Steps to Complete:

1. Document all security controls: access control (Keycloak), data encryption (at rest + transit), audit logging (PostgreSQL append-only), incident response (runbook), change management (CI/CD with approval gates)
2. Implement data retention policies: configure automatic deletion of conversations after 90 days (configurable per tenant), implement right-to-erasure API (GDPR): DELETE /v1/users/{id}/data deletes all PII
3. Implement audit log export: GET /v1/admin/audit/export → signed CSV download of audit_events for specified date range
4. Vulnerability management: configure Dependabot for automated dependency updates, set Trivy scanning to daily cron
5. Create compliance dashboard in Grafana: failed login attempts, data access by admin accounts, document deletion events, privilege escalation attempts
6. Conduct internal pen test: attempt to access another tenant's data, bypass HITL, extract secrets
7. Create security runbook docs/runbooks/security_incident.md

✅**Done When:** SOC2 evidence package complete. GDPR erasure endpoint tested. Compliance dashboard live. Internal pen test passed. Runbook reviewed by security team.

T-40 Phase 3 Validation [Phase 3] CRITICAL

Est. Time: 2 hours **Depends On:** T-32 through T-39

Steps to Complete:

1. Run full test suite: all unit, integration, E2E tests pass
2. Run DeepEval with memory tests: agent memory recall > 85% on cross-conversation facts
3. Validate Vault secret rotation under load (no 500 errors during rotation)
4. Validate Istio mTLS: use tcpdump to verify no plaintext traffic between services
5. Run chaos test: kill random pod every 30 seconds for 10 minutes — verify service remains available
6. Tag release v0.4.0-phase3

✅**Done When:** All tests pass. Memory recall > 85%. Secret rotation zero-downtime verified. mTLS confirmed. Chaos test survived. v0.4.0 released.

PHASE 4 — Distributed AI Platform — KubeRay, Multi-cluster

Month 9–12

T-42 KubeRay GPU Cluster Auto-Scaling [Phase 4] HIGH

Est. Time: 6 hours **Depends On:** All Phase 3

Steps to Complete:

1. Install KubeRay operator: helm install kuberay-operator kuberay/kuberay-operator
2. Create RayCluster manifest: 1 head node (4 CPU, 8G) + auto-scaling worker pool (GPU nodes, min 0, max 10)
3. Deploy vLLM on Ray: create Ray Serve deployment script that loads Mixtral 8x7B across Ray workers
4. Configure KEDA trigger on vLLM request queue depth: when pending requests > 10 for 2 minutes, KubeRay scales up a GPU worker
5. Implement GPU cost guard: when daily GPU spend exceeds threshold, route all requests to CPU model and alert
6. Benchmark: sustained 100 requests/minute — measure GPU auto-scale time, throughput, and cost

✅**Done When:** RayCluster auto-scales GPU workers under load. KEDA trigger fires correctly. GPU spend guard prevents runaway costs. 100 req/min handled with P95 < 1s.

T-43 Multi-Cluster Kubernetes (3-Region Active-Active) [Phase 4] HIGH

Est. Time: 10 hours **Depends On:** T-42

Steps to Complete:

1. Provision K8s clusters in 3 cloud regions using Terraform modules
2. Set up cluster federation: KubeFed or Argo CD ApplicationSet to deploy identical workloads to all 3 clusters
3. Configure global load balancer: AWS Global Accelerator / Cloudflare Load Balancing — route users to nearest healthy region
4. Configure cross-cluster PostgreSQL: one primary cluster, two replicas in other regions via logical replication
5. Configure cross-cluster Qdrant: replicate collections across regions for read scaling
6. Implement failover testing: kill entire primary region — verify traffic routes to secondary within 60 seconds
7. Run global latency test: measure P95 from 6 geographic locations — target < 200ms to nearest region

✅**Done When:** Traffic auto-routes to nearest region. Primary region failure triggers failover in < 60 seconds. Global P95 < 200ms from all test locations.

T-50 Agent Marketplace & Community Templates [Phase 4] MEDIUM

Est. Time: 8 hours **Depends On:** T-43

Steps to Complete:

1. Create marketplace/agents/ directory in repo with 15 production-ready agent templates
2. Build app/(app)/marketplace/page.tsx: agent template gallery with categories, ratings, one-click install
3. Create agent template format: agent_manifest.yaml (name, description, required_tools, RBAC_requirements, version)
4. Implement agent install flow: user browses marketplace, clicks Install, agent template added to their tenant
5. Build agent template submission: PR-based review process with DeepEval benchmark requirement (must show > 90% task success on benchmark)
6. Launch with 15 templates: HR Q&A, Legal Doc Analysis, Code Review, Sales Report, Support Ticket, Expense Report, Onboarding Guide, Contract Summary, Meeting Notes, RFP Response, Competitor Analysis, Bug Triage, API Doc Generator, Security Audit, Financial Summary

✅**Done When:** Marketplace shows 15 templates. One-click install works. Submitted templates pass DeepEval gate. Community PR template submission guide published.

T-51 Phase 4 Platform Completion Validation [Phase 4] CRITICAL

Est. Time: 3 hours **Depends On:** T-42 through T-50

Steps to Complete:

1. Run Locust: 10,000 concurrent users across 3 regions, P95 < 1s, error rate < 0.01%
2. KubeRay auto-scale test: ramp from 0 to 1000 GPU requests in 10 minutes — verify auto-scale without queue overflow
3. Multi-region failover test: kill primary region, time recovery, verify zero data loss
4. Run complete DeepEval suite: all benchmarks pass with final targets (Recall > 97%, Faithfulness > 95%, Hallucination < 1%)
5. Run Promptfoo full suite against all agents: zero successful attacks
6. Run full chaos test (Chaos Mesh): random pod kills for 1 hour — uptime > 99.99%
7. Tag release v1.0.0 — production ready
8. Publish to GitHub with comprehensive README, architecture docs, and video demo

✅**Done When:** 10,000 concurrent users served at P95 < 1s. All AI eval targets met. Zero security vulnerabilities. 99.99%+ uptime during chaos test. v1.0.0 released on GitHub.

Quick Reference — Task Summary by Priority

Complete Task Index (52 Tasks)

Task ID	Title	Phase	Priority	Est. Time
T-00	Initialize Monorepo & Dev Environment	Phase 0	CRITICAL	4 hrs
T-01	Docker Compose 6 Services	Phase 0	CRITICAL	3 hrs
T-02	FastAPI Service Skeleton	Phase 0	CRITICAL	4 hrs
T-03	PostgreSQL Schema & Alembic	Phase 0	CRITICAL	5 hrs
T-04	JWT Authentication	Phase 0	HIGH	3 hrs
T-05	Qdrant + Embedding Pipeline	Phase 0	CRITICAL	5 hrs
T-06	Document Upload & Ingestion	Phase 0	CRITICAL	6 hrs
T-07	Hybrid Search & RAG Retrieval	Phase 0	CRITICAL	5 hrs
T-08	LangGraph Agent + Chat Stream	Phase 0	CRITICAL	6 hrs
T-09	Next.js Chat UI + Doc Upload	Phase 0	CRITICAL	8 hrs
T-09B	DeepEval CI + README	Phase 0	HIGH	4 hrs
T-10	Keycloak Enterprise Auth	Phase 1	CRITICAL	5 hrs
T-11	LLM Guard Prompt Security	Phase 1	HIGH	4 hrs
T-12	Traefik API Gateway + TLS	Phase 1	HIGH	3 hrs
T-13	RAGFlow + DeepDoc Full Integration	Phase 1	HIGH	6 hrs
T-14	Temporal Workflow Engine	Phase 1	HIGH	6 hrs
T-15	Docker Swarm Production Deploy	Phase 1	HIGH	4 hrs
T-16	Prometheus + Grafana Observability	Phase 1	MEDIUM	5 hrs
T-17	Admin Panel + Document Library UI	Phase 1	MEDIUM	6 hrs
T-18	Hybrid BM25 + Dense Search Upgrade	Phase 1	HIGH	3 hrs
T-19	Phase 1 Review & Validation	Phase 1	CRITICAL	2 hrs
T-20	Kubernetes Setup (EKS/GKE/AKS)	Phase 2	CRITICAL	8 hrs
T-21	LangGraph HITL Full Implementation	Phase 2	CRITICAL	8 hrs
T-22	Composio MCP Gateway	Phase 2	HIGH	6 hrs
T-23	React Flow Workflow Builder	Phase 2	HIGH	8 hrs
T-24	NeMo Guardrails Agent Safety	Phase 2	HIGH	4 hrs
T-25	vLLM GPU Inference	Phase 2	MEDIUM	4 hrs
T-26	Canary Deployment Pipeline	Phase 2	HIGH	4 hrs
T-27	Agent Monitor + Analytics Upgrade	Phase 2	MEDIUM	5 hrs
T-28	Promptfoo Red-Teaming CI	Phase 2	HIGH	3 hrs

T-29	Tauri Desktop Client	Phase 2	MEDIUM	8 hrs
T-30	Phase 2 K8s Load Test Validation	Phase 2	CRITICAL	3 hrs
T-32	Graphiti Knowledge Graph Memory	Phase 3	HIGH	6 hrs
T-33	HashiCorp Vault Secrets	Phase 3	HIGH	5 hrs
T-34	Istio Ambient Service Mesh	Phase 3	MEDIUM	5 hrs
T-35	Multi-Agent CrewAI Teams	Phase 3	MEDIUM	6 hrs
T-36	SOC2 Audit Preparation	Phase 3	HIGH	10 hrs
T-40	Phase 3 Validation	Phase 3	CRITICAL	2 hrs
T-42	KubeRay GPU Auto-Scaling	Phase 4	HIGH	6 hrs
T-43	Multi-Cluster 3-Region Deploy	Phase 4	HIGH	10 hrs
T-50	Agent Marketplace	Phase 4	MEDIUM	8 hrs
T-51	Phase 4 Platform Validation + v1.0	Phase 4	CRITICAL	3 hrs

Total Estimated Build Time

Phase 0 (T-00 to T-09B): ~53 hours — 1 engineer working full-time = ~7 days
Phase 1 (T-10 to T-19): ~44 hours — 1–2 engineers = ~2–3 weeks
Phase 2 (T-20 to T-30): ~61 hours — 2–3 engineers = ~3–4 weeks
Phase 3 (T-32 to T-40): ~34 hours — 2 engineers = ~2 weeks
Phase 4 (T-42 to T-51): ~27 hours core tasks — 3 engineers = ~2 weeks
TOTAL: ~219 engineering hours — 8-engineer team: ~4 months to v1.0